Implementation Plan

IT Selaras Workspace — Fondasi Next.js Full-Stack + Supabase

Versi	Status	Tanggal	Dokumen Terkait
0.2	Menunggu Review	September 2026	SRS_IT_Selaras_Workspace.docx (requirement & keputusan akses D1–D4)

Dokumen ini menjabarkan BAGAIMANA dan KAPAN requirement pada SRS dibangun: pekerjaan tiap tahap, urutan pengerjaan, estimasi waktu, kriteria selesai (DoD), dan cara mengujinya. Pembagian fase pada v0.2 direvisi: permission (RLS + canDo()) dan activity log dibangun mulai Fase 1, sedangkan antarmuka pengelolaan anggota dan viewer audit log menyusul di Fase 3–4.

Ringkasan Eksekutif

Fondasi yang kuat di sini menentukan seberapa mudah tim menambahkan fitur selanjutnya tanpa refactor besar. Perubahan kunci pada v0.2:

- Keputusan kontrol akses D1–D4 (lihat SRS Bagian 3.1) diterapkan konsisten ke seluruh kode contoh (RLS, canDo()) di dokumen ini
- Permission enforcement dan activity logging dipindah ke Fase 1 — dibangun sejak fitur paling awal, bukan ditambahkan belakangan
- Halaman pengelolaan anggota (/admin/members) dan viewer audit log (/admin/audit-logs) tetap di Fase 3 dan 4 — hanya ANTARMUKA-nya yang menyusul, mekanismenya sudah berjalan sejak Fase 1
- Setiap fase kini memiliki estimasi waktu, urutan tugas, dan tabel kriteria selesai + cara menguji

Total Estimasi

Fase	Estimasi
Fase 0 — Setup & Fondasi	1–2 hari kerja
Fase 1 — Auth, Session, Permission & Activity Log Foundation	3–4 hari kerja
Fase 2 — Core Task & Project Management	4–6 hari kerja
Fase 3 — Panel Administrasi (Role Management)	2–3 hari kerja
Fase 4 — Activity Log Viewer & Polish	1–2 hari kerja
Review & Polish Akhir	1–2 hari kerja
Total	12–19 hari kerja (± 2,5–4 minggu, tergantung kecepatan tim)

1. Struktur Project

```
nextjs-stack/
├── src/
│   ├── app/ (auth)/ (dashboard)/ api/
│   ├── components/ ui/ layout/ tasks/ projects/
│   ├── lib/
```

```
│   │   ├── supabase/ client.ts server.ts
│   │   ├── auth/ session.ts permissions.ts   <-- canDo() dibangun Fase 1
│   │   └── activity/ logger.ts               <-- logActivity() dibangun Fase 1
│   ├── types/ database.ts auth.ts task.ts project.ts
│   ├── hooks/ useUser useTasks useProjects
│   └── middleware.ts
├── supabase/ migrations/ seed.sql
└── .env.local / .env.example / package.json
```

Konvensi penamaan dan environment variable mengikuti SRS Bagian 2.5 dan 2.6 — tidak diulang di sini agar tidak ada dua sumber kebenaran yang bisa berbeda.

2. Penerapan Keputusan Kontrol Akses (D1–D4) ke Kode

Keputusan D1–D4 ditetapkan di SRS Bagian 3.1. Bagian ini menerjemahkannya menjadi helper dan RLS yang konsisten.

2.1 Helper Permission (src/lib/auth/permissions.ts)

```
type Action = 'view' | 'create' | 'edit' | 'delete' | 'assign' | 'manage_members'
type Resource = 'project' | 'task'

interface ProjectMembership {
  role: 'owner' | 'member' | 'viewer' | null  // null = bukan member
}

interface TaskContext {
  createdBy: string
  assigneeId: string | null
}

export function canDo(
  user: { id: string; role: 'admin' | 'staff' },
  action: Action,
  resource: Resource,
  ctx: { membership?: ProjectMembership; task?: TaskContext }
): boolean {
  // D1 — Admin selalu menang di atas role project manapun
  if (user.role === 'admin') return true

  const membership = ctx.membership
  if (!membership || !membership.role) return false // bukan member -> selalu NO

  if (resource === 'project') {
    if (action === 'view') return true // owner/member/viewer semua bisa lihat
    // D3 — hanya owner yang bisa edit project / kelola member
    if (action === 'edit' || action === 'manage_members') return membership.role === 'owner'
    return false // create/delete project: admin only (sudah ditangani di atas)
  }

  if (resource === 'task') {
    if (action === 'view') return true
    if (action === 'create') return membership.role === 'owner' || membership.role === 'member'
    if (action === 'delete' || action === 'assign') return membership.role === 'owner'
    if (action === 'edit') {
      if (membership.role === 'owner') return true
      // D2 — member boleh edit HANYA jika creator atau assignee task tsb
      if (membership.role === 'member' && ctx.task) {
        return ctx.task.createdBy === user.id || ctx.task.assigneeId === user.id
      }
      return false // viewer tidak pernah bisa edit
    }
  }
```

```
  return false
}
```

2.2 RLS Policies Inti (menerapkan D1–D3)

```
-- PROJECTS: admin lihat semua (D1); member project lihat project miliknya
CREATE POLICY "Admin sees all projects" ON projects FOR SELECT
USING (EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'));

CREATE POLICY "Members see their projects" ON projects FOR SELECT
USING (id IN (SELECT project_id FROM project_members WHERE user_id = auth.uid()));

-- PROJECTS: hanya admin ATAU owner project yang boleh update (D3)
CREATE POLICY "Admin or owner can update project" ON projects FOR UPDATE
USING (
  EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin')
  OR EXISTS (
    SELECT 1 FROM project_members
    WHERE project_id = projects.id AND user_id = auth.uid() AND role = 'owner'
  )
);

-- TASKS: creator ATAU assignee boleh update (D2) -- plus admin & owner project
CREATE POLICY "Admin can do everything on tasks" ON tasks FOR ALL
USING (EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'));

CREATE POLICY "Owner can manage tasks in their project" ON tasks FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM project_members
    WHERE project_id = tasks.project_id AND user_id = auth.uid() AND role = 'owner'
  )
);

CREATE POLICY "Creator or assignee can update their task" ON tasks FOR UPDATE
USING (assignee_id = auth.uid() OR created_by = auth.uid());

-- TASKS: viewer & member boleh SELECT task dalam project mereka
CREATE POLICY "Members can view tasks in their project" ON tasks FOR SELECT
USING (project_id IN (SELECT project_id FROM project_members WHERE user_id = auth.uid()));

-- TASKS: hanya owner/member (bukan viewer) yang boleh INSERT task baru
CREATE POLICY "Owner or member can create tasks" ON tasks FOR INSERT
WITH CHECK (
  EXISTS (
    SELECT 1 FROM project_members
    WHERE project_id = tasks.project_id AND user_id = auth.uid()
      AND role IN ('owner', 'member')
  )
);
```

> **i** *Catatan desain: PostgreSQL RLS bersifat permissive OR antar-policy pada operasi yang sama — beberapa policy di atas sengaja dipecah per kondisi (admin / owner / creator-assignee) agar mudah diuji satu per satu (lihat Fase 2, kriteria pengujian RLS).*

2.3 Activity Logger (src/lib/activity/logger.ts) — dibangun Fase 1

```
export async function logActivity(params: {
  userId: string | null
  action: string
  resourceType: string
  resourceId?: string
```

```
  description: string
  metadata?: Record<string, unknown>
}) {
  // Dipanggil TANPA await di pemanggil (fire-and-forget) agar tidak
  // menghambat response utama -- NFR-PERF-01
  const supabase = await createClient()
  return supabase.from('activity_logs').insert({
    user_id: params.userId,
    action: params.action,
    resource_type: params.resourceType,
    resource_id: params.resourceId ?? null,
    description: params.description,
    metadata: params.metadata ?? null,
  })
}
```

3. Penanganan Sesi: Akun Dinonaktifkan & Akses Dicabut

Rincian teknis (masa berlaku token, perilaku signOut) sudah diverifikasi dan didokumentasikan di SRS Bagian 5.4–5.5. Bagian ini berisi implementasi konkretnya.

3.1 Pemeriksaan is_active di Session Helper

```
// src/lib/auth/session.ts
export async function requireAuth() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) redirect('/login')

  // WAJIB: jangan hanya percaya JWT valid -- is_active bisa berubah
  // kapan saja tanpa mencabut access token yang sudah terbit (lihat SRS 5.5-A)
  const { data: profile } = await supabase
    .from('profiles')
    .select('id, role, is_active')
    .eq('id', user.id)
    .single()

  if (!profile || !profile.is_active) {
    await supabase.auth.signOut()
    redirect('/login?reason=deactivated')
  }

  return { user, profile }
}
```

3.2 Memaksa Logout saat Admin Menonaktifkan Akun

```
// src/app/api/admin/members/[id]/route.ts  (PATCH)
// Dipanggil dengan service_role key, hanya di server
export async function PATCH(req: Request, { params }: { params: { id: string } }) {
  const { is_active } = await req.json()

  await supabaseAdmin.from('profiles').update({ is_active }).eq('id', params.id)

  if (is_active === false) {
    // Mencabut refresh token user tsb -- sesi tidak bisa diperpanjang lagi.
    // Access token yang sudah terbit TETAP valid sampai kedaluwarsa sendiri
    // (maks. 1 jam default) -- karena itu 3.1 di atas tetap wajib ada.
    await supabaseAdmin.auth.admin.signOut(params.id, 'global')

    await logActivity({
```

```
      userId: /* admin yang melakukan aksi */ null,
      action: 'USER_DEACTIVATED',
      resourceType: 'profile',
      resourceId: params.id,
      description: 'Akun dinonaktifkan oleh admin',
    })
  }

  return Response.json({ status: 'success' })
}
```

3.3 Revalidation Client saat Akses Project Dicabut

Karena RLS langsung berlaku di query berikutnya (SRS 5.5-B), fokus implementasi di client adalah memastikan data lama tidak "nyangkut" di layar:

- Gunakan staleTime pendek (atau revalidateOnFocus) pada hook data project/task sehingga perubahan keanggotaan tercermin saat user kembali ke tab
- Setelah mutasi membership (tambah/hapus member), invalidate cache query terkait project tsb di semua client yang relevan
- Tampilkan state "akses dicabut / project tidak ditemukan" yang jelas, bukan halaman kosong atau error generik, saat query RLS mengembalikan nol baris

4. Pembagian Fase, Urutan Pengerjaan, dan Pengujian

```
Fase 0: Setup & Fondasi                                    [1-2 hari]
    |
    v
Fase 1: Auth, Session, Permission & Activity Log Foundation [3-4 hari]  <- permission & log mulai di sini
    |
    v
Fase 2: Core Task & Project Management                      [4-6 hari]  <- canDo()+logActivity() DIPAKAI,
bukan dibangun ulang
    |
    v
Fase 3: Panel Administrasi (Role Management) -- UI menyusul [2-3 hari]
    |
    v
Fase 4: Activity Log Viewer -- UI menyusul                  [1-2 hari]
    |
    v
Review & Polish                                             [1-2 hari]
```

FASE 0 — Setup & Fondasi

Tujuan: Project siap untuk development; dependency terpasang; koneksi Supabase berfungsi; seluruh skema inti sudah ter-migrate sekaligus (bukan bertahap per fase) agar RLS Fase 1 bisa langsung dirancang menyeluruh.

Estimasi: 1–2 hari kerja

Tugas (urutan pengerjaan):

1. Install Supabase packages (@supabase/supabase-js, @supabase/ssr) dan Zod
2. Setup .env.local dari template .env.example
3. Buat Supabase project (dashboard atau local via `supabase start`)
4. Buat src/lib/supabase/client.ts (browser) dan server.ts (server)

5. Jalankan migration SQL seluruh skema inti sekaligus: profiles, projects, project_members (enum owner/member/viewer), tasks, activity_logs, beserta trigger handle_new_user()
6. Test koneksi: /api/health mengembalikan status DB
7. Update middleware dasar untuk session management (kerangka, logika lengkap di Fase 1)

Kriteria Selesai & Cara Menguji:

No	Kriteria Selesai (DoD)	Cara Menguji
1	npm run dev berhasil tanpa error	Jalankan `npm run dev`; pastikan tidak ada error di terminal maupun console browser
2	/api/health mengembalikan status Supabase connected	curl localhost:3000/api/health; cek response { status: "success", data: { supabase: "connected" } }
3	Supabase client bisa dibuat di server & browser	Import createClient() di satu Server Component dan satu Client Component; pastikan tidak ada error runtime
4	Seluruh tabel inti & enum sudah ter-migrate	Buka Supabase Table Editor (atau `\dt` + `\dT` di psql); pastikan 5 tabel dan 4 enum type ada
5	.env.example terdokumentasi lengkap	Review manual: bandingkan variabel di .env.local dengan .env.example, pastikan sinkron

FASE 1 — Auth, Session, Permission & Activity Log Foundation

Tujuan: User bisa login dengan aman; is_active divalidasi di setiap request; fondasi canDo() dan logActivity() SUDAH ADA dan diuji, meskipun belum dipakai di task/project (itu terjadi di Fase 2). Ini perubahan utama v0.2: permission & logging tidak menunggu sampai Fase 3/4.

Estimasi: 3–4 hari kerja (bertambah dari rencana sebelumnya karena permission & logging foundation dipindah ke sini)

Tugas (urutan pengerjaan):

1. Buat halaman /login dengan form email + password
2. Implementasi signInWithPassword via Supabase Auth
3. Implementasi logout: supabase.auth.signOut() + redirect
4. Buat src/lib/auth/session.ts: getUser(), requireAuth() — termasuk pemeriksaan profiles.is_active (lihat Bagian 3.1)
5. Update middleware.ts: redirect unauthenticated ke /login
6. Buat src/app/(dashboard)/layout.tsx dengan auth guard (requireAuth())
7. Tulis RLS dasar: profiles (user baca/ubah baris sendiri; admin baca semua), activity_logs (append-only, SELECT khusus admin)
8. Implementasi canDo() helper (Bagian 2.1) — unit test untuk kombinasi admin/owner/member(creator)/member(lain)/viewer
9. Implementasi logActivity() helper (Bagian 2.3), dipanggil asynchronous (fire-and-forget)
10. Terapkan logActivity() pada login/logout: USER_LOGIN, USER_LOGOUT, USER_LOGIN_FAILED

Kriteria Selesai & Cara Menguji:

No	Kriteria Selesai (DoD)	Cara Menguji
1	Login dengan kredensial valid ⟶ redirect ke /dashboard	Login manual via form; verifikasi URL berubah ke /dashboard

No	Kriteria Selesai (DoD)	Cara Menguji
2	Login salah → pesan error generik	Masukkan password salah; pastikan pesan tidak mengungkap "user tidak ditemukan" vs "password salah"
3	Akses /dashboard tanpa login → redirect /login	Buka /dashboard di browser incognito tanpa login
4	Logout menghapus session secara efektif	Logout, tekan tombol back browser ke /dashboard; harus redirect ke /login lagi
5	activity_logs mencatat USER_LOGIN / USER_LOGOUT	Query `select * from activity_logs order by created_at desc limit 5` setelah login/logout
6	Akun dinonaktifkan ditolak pada request berikutnya meski JWT belum kedaluwarsa	Set is_active=false langsung di DB saat user masih login; lakukan request berikutnya; harus ditolak sebelum 1 jam berlalu (NFR-SEC-06)
7	canDo() mengembalikan hasil benar untuk kombinasi role/aksi inti	Unit test canDo() minimal 5 skenario: admin, owner, member-creator, member-bukan-creator, viewer
8	logActivity() tidak menghambat response endpoint pemanggil	Ukur response time endpoint login dengan dan tanpa panggilan logActivity() (mis. dengan Chrome DevTools Network); selisih harus minimal

FASE 2 — Core Task & Project Management

Tujuan: User bisa melihat project sesuai akses (D1), melihat/membuat/mengubah task sesuai aturan creator-assignee (D2) dan role owner/member/viewer (D3). canDo() dan logActivity() dari Fase 1 DIPAKAI di sini, bukan dibangun ulang.

Estimasi: 4–6 hari kerja

Tugas (urutan pengerjaan):

1. Finalisasi & terapkan RLS lengkap pada projects, project_members, tasks (Bagian 2.2)
2. Generate TypeScript types dari schema: npx supabase gen types typescript
3. API: GET/POST /api/projects (list sesuai akses D1; create khusus admin)
4. API: GET/PATCH/DELETE /api/projects/[id] (edit: admin/owner; delete: admin)
5. API: POST/DELETE /api/projects/[id]/members (tambah/hapus member: admin/owner)
6. API: GET/POST /api/projects/[id]/tasks (create: admin/owner/member, bukan viewer)
7. API: GET/PATCH/DELETE /api/projects/[id]/tasks/[taskId] (edit: admin/owner selalu; member hanya jika creator/assignee — D2)
8. Setiap endpoint di atas memanggil canDo() sebelum mutasi dan logActivity() setelah sukses
9. Halaman /dashboard, /projects, /projects/[id]; komponen TaskCard, TaskList, TaskForm
10. Validasi input dengan Zod
11. Seed data: 1 admin, beberapa staff dengan kombinasi role owner/member/viewer di 2 project, 5 task dengan variasi creator/assignee

Kriteria Selesai & Cara Menguji:

No	Kriteria Selesai (DoD)	Cara Menguji
1	Admin melihat SEMUA project (D1)	Login sebagai admin; cek /projects menampilkan project yang admin bukan member-nya
2	Staff hanya melihat project tempat ia terdaftar	Login sebagai staff viewer di 1 project; pastikan project lain tidak muncul di /projects

No	Kriteria Selesai (DoD)	Cara Menguji
3	Owner dapat edit detail project & kelola member (D3)	Login sebagai owner (bukan admin); ubah nama project & tambah member; harus berhasil
4	Member biasa tidak bisa edit project	Login sebagai member; coba PATCH /api/projects/[id] langsung (curl/Postman); harus ditolak (403)
5	Viewer tidak bisa membuat/edit task	Login sebagai viewer; coba POST task; harus ditolak (403)
6	Creator DAN assignee sama-sama bisa edit task (D2)	Buat task sbg user A (creator), assign ke user B; login A edit (berhasil), login B edit (berhasil), login user C (member lain) edit (ditolak)
7	Semua perubahan tersimpan di Supabase dan konsisten setelah refresh	Lakukan perubahan, hard refresh halaman; data harus konsisten
8	activity_logs mencatat TASK_CREATED/UPDATED/STATUS_CHANGED dengan nilai lama & baru	Ubah status task; query activity_logs; cek field metadata berisi old & new status

FASE 3 — Panel Administrasi (Role Management)

> ℹ *Mekanisme permission 3-lapis dan logging untuk modul ini sudah berjalan sejak Fase 1 — fase ini HANYA membangun antarmuka (UI) dan endpoint pengelolaan role/status yang sebelumnya belum punya UI.*

Tujuan: Admin memiliki antarmuka untuk mengelola member, role global, role per-project, dan status aktif — tanpa perlu mengubah data langsung lewat Supabase dashboard.

Estimasi: 2–3 hari kerja

Tugas (urutan pengerjaan):

1. Buat halaman /admin/members (list member + role global + role per-project + status aktif)
2. API: GET /api/admin/members
3. API: PATCH /api/admin/members/[id] — ubah role global, role per-project, dan/atau is_active
4. Saat is_active diubah ke false: panggil supabase.auth.admin.signOut(userId, 'global') (Bagian 3.2)
5. Update middleware: redirect /admin/* untuk non-admin (memakai canDo() yang sudah ada sejak Fase 1)
6. logActivity() dipanggil pada endpoint baru ini: USER_ROLE_CHANGED, USER_ACTIVATED, USER_DEACTIVATED, PROJECT_MEMBER_ROLE_CHANGED

Kriteria Selesai & Cara Menguji:

No	Kriteria Selesai (DoD)	Cara Menguji
1	Menu Admin hanya tampil untuk role admin	Login sebagai staff; pastikan menu Admin tidak terlihat di sidebar
2	Akses /admin/* sebagai staff → redirect	Masukkan URL /admin/members manual sebagai staff; harus redirect ke /dashboard
3	Admin ubah role staff → admin berhasil	Ubah role via UI; verifikasi user tsb bisa akses /admin setelah re-login
4	Menonaktifkan akun langsung mencabut sesi (bukan menunggu token kedaluwarsa)	Nonaktifkan akun user yang sedang login; verifikasi refresh token user tsb sudah ditolak saat dicoba dipakai lagi
5	Semua aksi admin tercatat di activity_logs	Query activity_logs setelah tiap aksi admin; cek action code & metadata sesuai

FASE 4 — Activity Log Viewer & Polish

> ℹ *Pencatatan log sudah berjalan sejak Fase 1 (login/logout) dan Fase 2 (task/project). Fase ini membangun antarmuka untuk MENINJAU data yang sudah terkumpul, bukan membangun pencatatannya.*

Tujuan: Admin bisa meninjau riwayat aktivitas; UI rapi dan konsisten di seluruh aplikasi; flow end-to-end berjalan tanpa error.

Estimasi: 1–2 hari kerja

Tugas (urutan pengerjaan):

1. Halaman /admin/audit-logs dengan filter (user, action, tanggal)
2. API GET /api/admin/audit-logs dengan pagination
3. Review semua error state (403, 404, 500) di seluruh halaman
4. Pastikan semua halaman terproteksi punya loading state
5. Pastikan semua form punya feedback sukses/gagal
6. Review konsistensi UI (warna, spacing, typography)

Kriteria Selesai & Cara Menguji:

No	Kriteria Selesai (DoD)	Cara Menguji
1	Admin bisa melihat & memfilter log	Login admin, buka /admin/audit-logs, coba filter by user & by action; hasil sesuai
2	Pagination berfungsi tanpa duplikasi/lompat data	Seed >50 baris log; pastikan halaman berikutnya bisa diakses dan tidak ada data terlewat/duplikat
3	Tidak ada halaman crash saat akses ditolak	Akses endpoint terlarang dari role lain; harus tampil pesan 403 yang rapi, bukan crash/500
4	Flow utama end-to-end berjalan	Jalankan skenario penuh: login → lihat task → tambah task → update status → cek log tercatat, tanpa error

Review & Polish Akhir

Tujuan: Aplikasi siap direview sebagai satu kesatuan sebelum dianggap selesai fase MVP.

Estimasi: 1–2 hari kerja

Tugas (urutan pengerjaan):

1. Jalankan ulang seluruh skenario pengujian RLS (Fase 1 & 2) sebagai regresi akhir
2. Review README/setup instruction dengan developer baru (fresh clone)
3. Code review menyeluruh terhadap canDo() dan RLS untuk memastikan konsisten dengan D1–D4

Kriteria Selesai & Cara Menguji:

No	Kriteria Selesai (DoD)	Cara Menguji
1	Aplikasi bisa dijalankan di mesin fresh mengikuti README	Clone repo di mesin/akun baru; ikuti README dari nol tanpa bantuan tim
2	Tidak ada regresi pada aturan akses D1–D4	Jalankan ulang seluruh test case Fase 1 & 2 di atas satu per satu

5. Target Implementasi Pertama (MVP Awal)

Tercapai dengan menyelesaikan Fase 0 + Fase 1 + Fase 2 (± 8–12 hari kerja):

```
Login
  |
  v
Lihat Task sesuai akses (Admin: semua; Staff: sesuai project_members)
  |
  v
Tambah Task baru (Admin/Owner/Member, bukan Viewer)
  |
  v
Update status Task (Admin/Owner/creator/assignee)
  |
  v
Data tersimpan di Supabase
  |
  v
Aktivitas tercatat di activity_logs (sejak Fase 1, bukan baru di sini)
```

6. Risiko & Mitigasi

Risiko	Dampak	Mitigasi
RLS owner/member/viewer salah konfigurasi	Data leak antar user/project	Test RLS eksplisit tiap role per-project di Fase 2, regresi ulang di Review & Polish
Akun nonaktif tapi JWT lama masih diterima	Mantan user tetap bisa akses hingga 1 jam	is_active check di requireAuth() (Fase 1) + signOut paksa saat nonaktifkan (Fase 3)
Permission/log ditambahkan belakangan sehingga tidak konsisten	Technical debt, celah audit	canDo()/logActivity() dibangun & diuji di Fase 1, dipakai sejak Fase 2 (bukan ditambah di Fase 3/4)
Service Role Key ter-expose ke browser	Security breach	ESLint rule no-service-role-key-in-client
Type mismatch antara DB dan TypeScript	Runtime error	Generate types dari Supabase setelah setiap migration

7. Checklist Sebelum Development Dimulai

- Supabase project sudah dibuat (cloud atau local)
- Environment variables sudah dikonfigurasi di .env.local
- Semua anggota tim sudah membaca keputusan D1–D4 pada SRS Bagian 3.1
- Git branch strategy sudah disepakati
- SRS dan Implementation Plan ini sudah di-review dan disetujui

8. Riwayat Dokumen

Versi	Tanggal	Deskripsi Perubahan
0.1	September 2026	Draft awal
0.2	September 2026	Menyesuaikan dengan keputusan akses D1–D4 pada SRS (canDo()/RLS diperbarui); memindahkan permission & activity log ke Fase 1; menambahkan estimasi waktu per fase dan tabel kriteria

Versi	Tanggal	Deskripsi Perubahan
		selesai + cara menguji; menambahkan penanganan akun dinonaktifkan/akses dicabut saat sesi aktif.

Dokumen ini akan diupdate setelah review dan selama proses implementasi.